Flutter Documentation: Hide BottomNavigationBar on Scroll

Overview

This pattern hides the `BottomNavigationBar` when the user scrolls **down** and shows it again when the user scrolls **up**.

This improves UX by giving more screen space for content while scrolling.

Key Concepts Used

1. NotificationListener

Listens to scroll notifications from scrollable widgets like `ListView`.

```
NotificationListener<ScrollNotification>
```

It allows reacting to user scroll behavior.

---

2. UserScrollNotification

This specific notification tells us the **direction of scrolling**.

Possible directions:

Direction	Meaning
ScrollDirection.forward	User scrolling UP
ScrollDirection.reverse	User scrolling DOWN

---

3. ScrollDirection

Imported from:

```
import 'package:flutter/rendering.dart';
```

Used to detect scroll direction.

---

4. AnimatedContainer

Used to animate the bottom bar height when hiding or showing.

```
AnimatedContainer(
  duration: Duration(milliseconds: 100),
  height: _isVisible ? kBottomNavigationBarHeight : 0,
)
```

If `_isVisible` is:

Value	Result
true	Bottom bar visible
false	Bottom bar hidden

Logic Flow

```
User Scrolls
     ↓
NotificationListener detects scroll
     ↓
Check ScrollDirection
     ↓
If reverse (scroll down)
     ↓
Hide BottomNavigationBar

If forward (scroll up)
     ↓
Show BottomNavigationBar
```

Step-by-Step Logic

Step 1

Create a state variable controlling visibility

```
bool _isVisible = true;
```

Step 2

Listen to scroll notifications

```
NotificationListener<ScrollNotification>
```

Step 3

Check if notification is a user scroll

```
if (scrollNotification is UserScrollNotification)
```

Step 4

Detect scroll direction

```
scrollNotification.direction
```

Step 5

Hide bottom navigation when scrolling down

```
if (scrollNotification.direction == ScrollDirection.reverse) {
  if (_isVisible) setState(() => _isVisible = false);
}
```

Step 6

Show bottom navigation when scrolling up

```
else if (scrollNotification.direction == ScrollDirection.forward) {
  if (!_isVisible) setState(() => _isVisible = true);
}
```

Full Source Code

```
import 'package:flutter/rendering.dart';
import 'package:flutter/material.dart';

class ScrollHideBottomNavExample extends StatefulWidget {
  const ScrollHideBottomNavExample({super.key});

  @override
  _ScrollHideBottomNavExampleState createState() =>
      _ScrollHideBottomNavExampleState();
}

class _ScrollHideBottomNavExampleState
    extends State<ScrollHideBottomNavExample> {
  bool _isVisible = true;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: NotificationListener<ScrollNotification>(
        onNotification: (scrollNotification) {
          if (scrollNotification is UserScrollNotification) {
            if (scrollNotification.direction == ScrollDirection.reverse) {
              if (_isVisible) setState(() => _isVisible = false);
            } else if (scrollNotification.direction == ScrollDirection.forward)
{
              if (!_isVisible) setState(() => _isVisible = true);
            }
          }
          return true;
        },
        child: ListView.builder(
          itemCount: 100,
          itemBuilder: (context, index) =>
```

```
              ListTile(title: Text('Item $index')),
        ),
      ),
      bottomNavigationBar: AnimatedContainer(
        duration: Duration(milliseconds: 100),
        height: _isVisible ? kBottomNavigationBarHeight : 0,
        child: Wrap(
          children: [
            BottomNavigationBar(
              items: const [
                BottomNavigationBarItem(
                    icon: Icon(Icons.home), label: "Home"),
                BottomNavigationBarItem(
                    icon: Icon(Icons.qr_code), label: "QR"),
              ],
            ),
          ],
        ),
      ),
    );
  }
}
```

Advantages of This Pattern

- Better screen space while scrolling
- Smooth UI animation
- Common in modern apps

Examples:

- Instagram
- Twitter/X
- YouTube

Possible Improvements

You can improve this widget by:

- Adding `AnimationController`
- Using `AnimatedSlide`
- Using `ScrollController` instead of NotificationListener

- Adding blur effect when appearing

End of documentation.